

Differences Between UNION EXCEPT and INTERSECT Operators in SQL Server

In this article, I will explain the **differences between UNION EXCEPT and INTERSECT Operators in SQL Server** with some examples. Please read [UNION and UNION ALL](#), [EXCEPT](#), and [INTERSECT](#) Operators of this article series before proceeding to this article. The SET operators are mainly used to combine the result of more than 1 select statement and return a single result set to the user.

The set operators work on complete rows of queries, so the results of the queries must have the same column name and column order, and the types of columns must be compatible. There are the following 4 set operators in SQL Server:

1. **UNION**: Combine two or more result sets into a single set without duplicates.
2. **UNION ALL**: Combine two or more result sets into a single set, including all duplicates.
3. **INTERSECT**: Takes the data from both result sets, which are in common.
4. **EXCEPT**: Takes the data from the first result set but not in the second result set (i.e., no matching to each other)

Rules on Set Operations:

1. The result sets of all queries must have the same number of columns.
2. In every result set, the data type of each column must be compatible (well-matched) with the data type of its corresponding column in other result sets.
3. An ORDER BY clause should be part of the last select statement to sort the result. The first select statement must find out the column names or aliases.

Understand the Differences Between These Operators with Examples.

Use the SQL Script to create and populate the two tables we will use in our examples.

```
CREATE TABLE TableA
(
    ID INT,
    Name VARCHAR(50),
    Gender VARCHAR(10),
    Department VARCHAR(50)
)
GO
INSERT INTO TableA VALUES(1, 'Pranaya', 'Male', 'IT')
INSERT INTO TableA VALUES(2, 'Priyanka', 'Female', 'IT')
INSERT INTO TableA VALUES(3, 'Preety', 'Female', 'HR')
INSERT INTO TableA VALUES(3, 'Preety', 'Female', 'HR')
GO
```

Fetch the records:

```
SELECT * FROM TableA
```

ID	Name	Gender	Department
1	Pranaya	Male	IT
2	Priyanka	Female	IT
3	Preety	Female	HR
3	Preety	Female	HR

```

CREATE TABLE TableB
(
    ID INT,
    Name VARCHAR(50),
    Gender VARCHAR(10),
    Department VARCHAR(50)
)
GO
INSERT INTO TableB VALUES(2, 'Priyanka', 'Female','IT')
INSERT INTO TableB VALUES(3, 'Preety', 'Female','HR')
INSERT INTO TableB VALUES(4, 'Anurag', 'Male','IT')
GO

```

Fetch the records:

SELECT * FROM TableB

ID	Name	Gender	Department
2	Priyanka	Female	IT
3	Preety	Female	HR
4	Anurag	Male	IT

UNION Operator:

The Union operator will return all the unique rows from both queries. Notice that the duplicates are removed from the result set.

SELECT ID, Name, Gender, Department FROM TableA
UNION
SELECT ID, Name, Gender, Department FROM TableB

Result:

ID	Name	Gender	Department
1	Pranaya	Male	IT
2	Priyanka	Female	IT
3	Preety	Female	HR
4	Anurag	Male	IT

Purpose: The UNION operator combines the result sets of two or more SELECT statements into a single result set.

Distinct Values: It removes duplicate rows between the various SELECT statements.

Use Case: You would use UNION when listing all distinct rows from multiple tables or queries.

UNION ALL Operator:

The UNION ALL operator returns all the rows from both queries, including the duplicates.

```
SELECT ID, Name, Gender, Department FROM TableA
UNION ALL
SELECT ID, Name, Gender, Department FROM TableB
```

Result:

ID	Name	Gender	Department
1	Pranaya	Male	IT
2	Priyanka	Female	IT
3	Preety	Female	HR
3	Preety	Female	HR
2	Priyanka	Female	IT
3	Preety	Female	HR
4	Anurag	Male	IT

INTERSECT Operator:

The INTERSECT operator retrieves the common unique rows from the left and right queries. Notice the duplicates are removed.

```
SELECT ID, Name, Gender, Department FROM TableA
INTERSECT
SELECT ID, Name, Gender, Department FROM TableB
```

Result:

ID	Name	Gender	Department
2	Priyanka	Female	IT
3	Preety	Female	HR

Purpose: The INTERSECT operator returns all rows common to both SELECT statements.

Distinct Values: Like UNION and EXCEPT, INTERSECT also removes duplicates.

Use Case: You would use INTERSECT when you need to find rows that are shared between two tables or queries.

EXCEPT Operator:

The EXCEPT operator will return unique rows from the left query that aren't in the right query's results.

```
SELECT ID, Name, Gender, Department FROM TableA
EXCEPT
SELECT ID, Name, Gender, Department FROM TableB
```

Result:

ID	Name	Gender	Department
1	Pranaya	Male	IT

If you want the rows that are present in Table B but not in Table A, reverse the queries.

```
SELECT ID, Name, Gender, Department FROM TableB
EXCEPT
SELECT ID, Name, Gender, Department FROM TableA
```

Result:

ID	Name	Gender	Department
4	Anurag	Male	IT

Purpose: The EXCEPT operator returns all rows from the first SELECT statement that are absent in the second SELECT statement's results.

Distinct Values: It automatically removes duplicates.

Use Case: EXCEPT is used when you want to find rows in one query that are not found in another. It's useful for finding differences between tables or queries.

Note: For all these 4 operators to work, the following 2 conditions must be met

1. The number and the order of the columns must be the same in both the queries
2. The data types must be the same or at least compatible

For example, if the number of columns is different, you will get the following error

Msg 205, Level 16, State 1, Line 1

All queries combined using a UNION, INTERSECT, or EXCEPT operator must have an equal number of expressions in their target lists.

Differences Between UNION EXCEPT and INTERSECT Operators in SQL Server:

In SQL Server, the UNION, EXCEPT, and INTERSECT operators combine or manipulate the results of two or more SELECT statements. These operators help you perform set operations on the result sets of those SELECT statements. Here are the main differences between these operators:

UNION Operator:

1. The UNION operator combines the result sets of two or more SELECT statements into a single result set.
2. It removes duplicate rows from the combined result set by default.
3. The columns in the SELECT statements must have compatible data types, and the number of columns in each SELECT statement must be the same.

4. The order of rows in the final result set may not be the same as in the individual SELECT statements unless you use the ORDER BY clause.

EXCEPT Operator:

1. The EXCEPT operator retrieves the rows present in the first result set but not in the second result set.
2. It returns distinct rows from the first result set that do not have corresponding rows in the second result set.
3. The columns in both SELECT statements must have compatible data types, and the number of columns in both statements must be the same.

INTERSECT Operator:

1. The INTERSECT operator is used to retrieve the rows that are common to both result sets.
2. It returns distinct rows appearing in the first and second result sets.
3. The columns in both SELECT statements must have compatible data types, and the number of columns in both statements must be the same.

So, UNION combines result sets, EXCEPT returns rows from the first set that are not in the second set, and INTERSECT returns common rows between two result sets. It's important to ensure that the data types and the number of columns match when using these operators, and you can use the ORDER BY clause to control the order of the final result set if needed.

In the next article, I will discuss the [Joins in SQL Server](#) with Examples. In this article, I try to explain the **differences between UNION EXCEPT and INTERSECT Operators in SQL Server** with some examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this article.